本章路線

這章要回答的核心問題是：在任意交錯下，如何保證共享狀態正確，同時保留進度與等待上限？

前置知識。 process 與 thread 狀態、共享記憶體、交錯與原子操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 證明 mutual exclusion、progress 與 bounded waiting
- 用 semaphore 建立明確順序與背壓
- 辨識 deadlock、starvation、priority inversion 與 lost wake-up

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1 Background

- race condition 發生於多個執行流的存取結果取決於未受控交錯。只要 shared mutable state 缺乏共同同步規則，就可能出錯。
- atomic operation 是執行期間不可被觀察為一半的基礎。硬體提供原子指令，高階 API 再建立較容易使用的鎖與條件等待。

Critical section 四段



Mutual exclusion	最多一個執行流在 critical section
Progress	候選者存在時必須有人能決定是否進入
Bounded waiting	任何等待者不能被無限次跳過

5.1 模型、時序與反例：Background

模型假設。 shared mutable state 可被多執行流存取；每個記憶體操作有架構定義的原子性與可見性規則。

不變量與時序。 所有可觀察的中間狀態要嘛保有 shared invariant，要嘛只存在由鎖或交易隔離的區域。

反例或邊界。 兩個 threads 分別更新 linked list 的 next 與 size，即使各自單一指令原子，整體結構仍可能不一致。

完整算例。 兩個 writers 對 balance=100 各加 50，若都讀舊值再寫回，最終 150 而非 200。

思考題。 為何單一原子指令不一定足以保護多欄位 invariant？

5.1　race condition 的共同不變量

兩個執行流競爭同一份可寫資料時，結果可能取決於載入、修改、寫回的發生順序。把 `counter++` 看成單一步驟會漏掉真正的交錯。

令 producer 執行 `R1 = counter`、`R1 = R1 + 1`、`counter = R1`，consumer 執行 `R2 = counter`、`R2 = R2 - 1`、`counter = R2`。兩個執行流各自成立時 counter 不變；任意交錯時可能得到 4、5 或 6。

不變量的模型邊界。 mutual exclusion 只保護程式約定的區段。記憶體可見性、編譯器重排、非原子寬度與失敗後重試，仍要由同一個同步契約交代。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1　逐步交錯：為何 counter=4

初始 `counter=5`。下列交錯保留每個高階指令內部的順序，但允許兩個執行流穿插。

事件	Producer	Consumer	counter	說明
T0	`R1=5`		5	producer 讀舊值
T1	`R1=6`		5	結果只在暫存器
T2		`R2=5`	5	consumer 也讀舊值
T3		`R2=4`	5	結果只在暫存器
T4	`counter=6`		6	producer 寫回
T5		`counter=4`	4	consumer 覆蓋

交換 T4、T5 會留下 6。正確結果是 5，因為 producer 的新增與 consumer 的移除互相抵銷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1 何時需要同步

1. 列出所有 shared mutable locations。
2. 寫出跨欄位仍需成立的不變量，例如 `0 <= size <= capacity` 或 `readers=0 or writer=0`。
3. 找出每個會暫時破壞不變量的多步驟操作。
4. 定義哪些觀察者不能在操作中途看見狀態，並指定 acquire、release 與可見性規則。

原子指令只保護單一硬體操作。若更新 linked list 的 `next` 與 `size`，即使每次 store 都原子，兩個欄位之間仍可被觀察為不一致。atomicity 是起點，不足以自動保護複合不變量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.2 Critical-section problem

- mutual exclusion 保證任何時刻最多一個執行流位於 critical section。progress 保證 critical section 空著且有人等待時，非 remainder 的候選者能在有限步內決定下一位，不能無限期延後。
- bounded waiting 為每個請求者規定有限上界，限制它獲准前可被多少其他執行流插隊。三性質分別處理安全、整體活性與個別公平；只有 bounded waiting 能排除個別 thread 被無限跳過。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.2　模型、時序與反例：Critical-section problem

模型假設。 候選者可能在任意相對速度交錯；進入與離開區段有可觀察協議，等待不會假裝完成。

不變量與時序。 進入門檻必須讓 mutex 安全成立；只要有候選者，離開者就必須提供可持續前進的路徑。

反例或邊界。 無限輪轉只讓較高優先權者進入，雖有 progress，低優先權者卻沒有 bounded waiting。

完整算例。 counter 初值 0，P1/P2 各執行 load、add 1、store。交錯 P1.load=0、P2.load=0、P1.store=1、P2.store=1，兩次 inc 後 counter 仍是 1，證明 lost update；把 load/add/store 包進互斥區，重跑得到 2。

思考題。 三性質各對應哪一類反例，能否互相推出？

5.2　三個要求的精確界線

有 n 個程序 `{P0,...,P(n-1)}`。每個程序依序有 entry section、critical section、exit section、remainder section。

要求	完整定義	失敗會看到什麼
Mutual exclusion	若 `Pi` 正在 critical section，其他程序不得同時在各自的 critical section	兩個 writer 同時修改同一不變量
Progress	critical section 空著，且有人想進入時，只有非 remainder 的候選者參與選取，而且選取不可無限期延後	所有人都互相等待，或一直沒有人被選中
Bounded waiting	對每個請求者存在有限界線，限制在它獲准前允許多少其他程序進入	系統持續有進展，但某個固定程序一直輪不到

三項要求回答不同問題，任何一項都不能由另兩項自動推出。

5.2　三個彼此獨立的反例

只有 mutual exclusion。 一個不公平的鎖永遠讓高優先權者插隊。任何時刻仍只有一人進入，但低優先權者沒有 finite bound。

有 mutual exclusion 與 progress，沒有 bounded waiting。 每個空檔都有人被選中，整體進展存在，特定 thread 卻被無限次插隊。這是 starvation。

沒有 mutual exclusion。 所有人可能同時通過 entry section。即使平均等待時間很短，安全不變量仍立即失效。

證明的最小義務。 安全性質要覆蓋所有可達交錯。進度與公平要另行證明：沒有等待中的死點，且每個等待者存在有限上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.2　entry 與 exit 的檢查表

對協議 `enter_i / leave_i` 檢查四件事：

1. 假設任意相對速度，仍不得有兩個執行流同時在 critical section。
2. 若 critical section 空著且存在候選者，至少一個候選者或離場者在有限步內使某人可進入。
3. 每個請求者在有限次他人進入前取得許可。
4. entry 與 exit 自身使用什麼原子操作、記憶體順序與失敗恢復。

只測幾個排程順序不足以證明；有限測試只能找反例，不能涵蓋 `n` 個程序的任意速度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3 Peterson's solution

- Peterson 的雙程序解使用 flag 表示想進入，turn 裁決同時想進入的情況。
- 它在順序一致的共享讀寫假設下提供互斥、progress 與 bounded waiting。現代弱記憶體模型若不補 fencing 或原子操作，不能直接假定成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3 模型、時序與反例：Peterson's solution

模型假設。 兩個程序、共享 flag 與 turn 具有 sequential consistency；讀寫不會被重排成演算法未涵蓋的交錯。

不變量與時序。 同時想進入時 turn 只選一方；等待者保留 flag，使離場者必須讓出下一次選擇。

反例或邊界。 在弱記憶體模型中，沒有 fence 的 flag/turn 寫入可見順序可能不同，原證明前提失效。

完整算例。 P0 先寫 flag0=1、turn=1，若 P1 尚未 competing，P0 看見 flag1=0 而進入。若 P1 隨後寫 flag1=1、turn=0，它看到 flag0=1 且 turn=0，於是等待。P0 離場令 flag0=0 後，P1 可進；P0 若再 competing 必先寫 turn=1，因此 P1 至多被 P0 插隊一次。若兩人在此之前都有 flag=1，則 turn=0 或 turn=1，恰有一方的 while 條件不成立。

思考題。 若移除 sequential consistency 假設，哪一步需要額外硬體或編譯器保證？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3 Peterson 演算法的完整協議

Peterson 演算法只處理兩個程序。令 `j=1-i`，共享 `boolean flag[2]={false,false}` 與 `int turn`。

```
P_i:
  flag[i] = true
  turn = j
  while (flag[j] && turn == j) { /* wait */ }
  critical-section_i
  flag[i] = false
  remainder_i
```

`flag[i]` 表示 Pi 正在 competing。`turn` 不安裝永久 lock；它只裁決雙方同時 competing 時，誰先退讓。因此 `turn` 的值不能直接解釋成持有權，更不能推論「最後寫 turn 的人必敗」。

5.3 互斥性證明

固定shared reads/writes皆為atomic且sequentially consistent。反設P0、P1本輪critical sections重疊；令W0為P0本輪的`turn=1`，W1為P1本輪的`turn=0`。總順序下只有兩種可能。

1. 若W0先於W1，P0的`flag0=true`又先於W0，因此在W1之前已成立。P1要在P0離開本輪CS前進入，就必須在flag0仍true時通過while；此時最後turn寫入是W1的0，故P1條件`flag0 && turn==0`為true，不能進入，矛盾。
2. 若W1先於W0，對稱地P0看到`flag1=true && turn==1`而受阻，也矛盾。

比較的是這兩次相互重疊嘗試的最後turn寫入；兩者在本輪CS內都不再寫turn或清flag，直到離場才清自己flag。若有一方根本未compete，另一方可因對方flag=false進入，不應把「最後寫turn必敗」外推成不附條件的口號。

5.3 progress、有限等待與記憶體順序

Progress。 若只有一人 competing，另一人的 flag 為 false，前者可進。若兩人 competing，turn 只能是 0 或 1，所以恰有一人在其 while 上不因 turn 受阻。若另一人已在 critical section，它完成後會清除 flag，不能讓第三人妨礙這個二人集合。

Bounded waiting。 Pi 被擋時，Pj 必為 `flag[j]=true && turn=j`。Pj 在 critical section 中不改 turn。Pj 離場後清除 flag；若它要再次插隊，必先重寫 `turn=i`，這會使 Pi 優先。因此 Pi 至多等 Pj 一次進出。

前提。 原證明假設 shared reads/writes 具有需要的順序與可見性。現代 C/C++ 實作應使用相同位置的 atomic operations 與 `memory_order_seq_cst`，或建立等價 fence；只把一般變數改名成 flag 不提供證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.4 同步硬體

- test-and-set 或 compare-and-swap 可原子地測試並設值。spinlock 以 busy waiting 等待，較適合臨界區短、持鎖者能在另一核心繼續執行的 SMP 情況。
- spinlock 會消耗 CPU，甚至可能阻礙持鎖者執行。單核、長臨界區、interrupt context 或高競爭情況下要特別處理，原子指令也不代表整個 critical section 自動原子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.4　模型、時序與反例：同步硬體

模型假設。 test-and-set 或 compare-and-swap 依架構規則原子，spin 期間持鎖者最終可執行。

不變量與時序。 只有成功完成原子狀態轉移者持有鎖；release 後下一次 acquire 能觀察更新。

反例或邊界。 單核上若 interrupt handler 自旋等待被中斷程式持有的同一鎖，owner 無法恢復而死結；搶占式優先權排程也可能讓 spinner 阻礙 owner。這不能描述成 nonpreemptive 卻假定 owner 被排程搶占。

完整算例。 兩個核心競爭 lock=0，CAS 只讓一個成功；失敗者先執行 limited backoff 再重試。

思考題。 何時 spin 比 sleep/wake 更便宜，何時會浪費整個核心？

5.4　test-and-set 與 compare-and-swap

`test_and_set(&L)` 原子地回傳舊值，並把 L 設為 true。`compare_and_swap(&L,0,1)` 只有在 L 等於 0 時寫 1，且無論成功與否都回傳舊值。

```
acquire(L):
  repeat
    old = test_and_set(L)
  until old == false

release(L):
  L = false
```

第一次 acquire 看見 `old=false` 並持有 L。其他執行流看見 `old=true` 而重試。只要所有 release 都公開寫回 false，mutual exclusion 成立。

5.4 spinlock 的邊界

原始 test-and-set 鎖不保證 bounded waiting。失敗者可能反覆搶輸，某個執行流沒有有限上界。公平鎖要再排隊，例如把 waiting array 掃成環，離場者指定下一個等待者。

成本。 spin 的執行流不 sleep，所以短臨界區可省下 sleep/wakeup 與 context switch。長臨界區、單核，或 spinner 與 owner 競爭同一個核心時，spin 會浪費 CPU。

單核中斷的危險。 若中斷處理器自旋等待被中斷程式持有的同一把鎖，owner 在中斷返回前無法執行，可能形成 live lock 或 deadlock。中斷上下文需要限制 spin 條件或使用不會 sleeping 的受控 primitive。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.4 atomic 不等於跨平台 acquire/release

硬體交易只保證指令本身不可分割。高階語言還要保證：

- acquire 後可見 owner 在 release 前寫入的資料。
- release 前的一般資料寫入不會被搬到 release 之後。
- compiler 不會把 critical section 讀寫重排到鎖外。

因此程式使用 lock 的 atomic acquire/release semantics 或對應 fence，而不是只用一般讀寫拼一個旗標。CAS 也無法單獨處理 lock-free 資料結構的 ABA、節點回收與 progress guarantee。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.5 Mutex locks

- mutex 提供 acquire 與 release 的互斥契約。阻塞式 mutex 將等待者休眠，通常比長期 spin 節省 CPU。
- lock ordering 可避免某些 deadlock。所有持有關係都必須在例外與取消路徑正確釋放。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.5 模型、時序與反例：Mutex locks

模型假設。 mutex 有 owner、每次 acquire 對應一次 release，等待者可被喚醒且喚醒後重新競爭。

不變量與時序。 protected data 的存取都在持有同一 mutex 時發生；鎖釋放後不得有未完成的資料結構更新。

反例或邊界。 兩個函式以相反順序取得 A、B，即使每次只持有兩鎖之一，仍可能形成等待環。

完整算例。 固定 lock id A=1、B=2，且每個 transaction 只依遞增順序取得。若 T1 持 A 等 B，T2 可能持 B 並已請求 A，但這個 B→A 請求違反順序，協議會在 acquire 前阻止它。因此不存在 T1→T2 與 T2→T1 同時成立；有限次執行只作為實作檢查，一般性結論來自順序反證。

思考題。 exception 或 cancel 路徑如何保證 release 一定發生？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.5 mutex 的契約

每個 critical section 必須和同一把 mutex 綁定：

```
acquire(m)
critical section
release(m)
```

acquire 與 release 本身必須是 atomic operation。mutex 有 owner 時，通常由同一 owner release；semaphore 則沒有這種 owner 檢查。這個差異會影響錯誤路徑、取消與異常處理。

阻塞式 mutex。 競爭者進入 waiting state，release 時喚醒一個等待者。**spinlock。** 競爭者留在 CPU 上重試，只適合短且 owner 確能執行的區段。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.5 lock ordering 的數學理由

定義嚴格全域順序 `ord(A)=1`、`ord(B)=2`。協議要求每個執行流只在手中最大順序小於新鎖時取得新鎖。

若 A→B 的等待環存在，必有一條邊 B→A 表示某執行流持 B 卻取得 A。這要求 `ord(B) < ord(A)`；同一協議又要求參與循環的所有順序遞增，接回起點後得到 `ord(A) < ... < ord(A)`，矛盾。

實作條件。 callback、condition wait、lock upgrade、例外與取消路徑都要遵循同一順序。動態物件要先以穩定 key 排序，例如 address 或唯一 id，再依排序取得兩把鎖；直接按 transaction 參數順序取得 from/to 兩帳戶鎖仍可能反向。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.6 Semaphores

- counting semaphore 表示可用資源數，binary semaphore 可表示事件或互斥。wait 減值後若小於零阻塞，signal 增加並可能喚醒等待者。
- semaphore 未綁定 owner 時容易在錯誤路徑 signal 太多次，破壞資源計數。程式必須清楚標示誰負責 wait 與 signal。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.6 模型、時序與反例：Semaphores

模型假設。 semaphore 初值表示資源數，wait/signal 各為不可分割操作，等待者可在資源可用時被喚醒。

不變量與時序。 固定 C 個資源的可用數加上已配置數必須等於 C。signed semaphore 的內部值可為負以表示等待者，不等於實體資源為負；事件型 semaphore 也不必以上述初值為上界。

反例或邊界。 consumer 應先等待 full，producer 應先等待 empty，再取得 buffer mutex。若 producer 持有 mutex 等待空槽，consumer 無法取出項目，形成資源與鎖等待環。

完整算例。 capacity 3 時 empty=3、full=0，三個 produce 後 full=3，第四個 producer 在 empty 上阻塞。

思考題。 semaphore 的無 owner 特性帶來哪些錯誤路徑風險？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.6 semaphore 的兩種實作

資源型counting semaphore以permit表示資源；binary抽象只容許free permit為0或1。下列blocking實作使用signed counter記等待者，內部值可為負，不能和binary抽象狀態混用。wait/signal的counter與queue操作必須原子銜接。

busy-wait 定義。

```
wait(S):   while (S <= 0) {} ; S--
signal(S): S++
```

blocking 定義。

```
wait(S):
  S--
  if (S < 0) { add current process to S.queue; block(); }

signal(S):
  S++
  if (S <= 0) { remove one process from S.queue; wakeup(it); }
```


5.6 semaphore 的兩種實作 （續）

在blocking實作中，負值表示等待者而不是負資源。靜止邊界下free permits（available=max(S,0)）、queued waiters=max(−S,0)；handoff已授出的permit算allocated，即使該waiter尚未實際得到CPU，也不能再被新waiter取得。資源模型檢查free+allocated=capacity；事件型semaphore則不任意套固定容量。

容量1的完整交接：T0 wait使1→0取得permit；T1 wait使0→−1並睡眠；T0 signal使−1→0，因加後S<=0而wake T1並handoff；T1恢復原wait之後的程式，不再減S。T1 signal使0→1且無waiter，這次才留下free permit。若只在0→1時wake，會使T1永遠睡眠。

5.6　事件語意與 bounded-buffer 順序

令 `empty=3`、`full=0`、`mutex=1`，buffer 為空。下列是合法事件的前後狀態。

已完成的操作	empty	full	buffer	可觀察事實
初始	3	0	`[]`	三個空槽
put A	2	1	`[A]`	一項已發布
put B	1	2	`[A,B]`	`empty+full=3`
put C	0	3	`[A,B,C]`	下次 put 必須等待
get A	1	2	`[B,C]`	取出後釋放容量
put D	0	3	`[B,C,D]`	容量仍為 3

producer 應 `wait(empty); wait(mutex); ...; signal(mutex); signal(full)`。consumer 對稱地先 `wait(full)`。先取得 mutex 再等資源可能持鎖睡眠，令能產生該資源的對方也卡在 mutex。

5.6　死結、飢餓與 priority inversion

設 S、Q 初值皆為 1。P0 做 `wait(S); wait(Q)`，P1 做 `wait(Q); wait(S)`。若 P0 先取得 S、P1 先取得 Q，兩者接著等待對方持有的 semaphore，沒有第三者能 signal，形成 deadlock。

可執行但長期不被喚醒是 starvation。LIFO semaphore queue 可能使先抵達者永遠被新到達者越過；FIFO 只是避免一種 starvation 的設計，不能自動修復配對錯誤。

Priority inversion。低優先權 L 持有 H 所需資源時，若 M 可搶占 L，H 就會被間接延後。priority inheritance 讓 L 暫時升到 H 的優先權，完成資源區段後恢復原值。MARS Pathfinder 的 VxWorks 系統即以此修復高優先權任務被中優先權任務延後的事件。

5.7　經典同步問題

- bounded-buffer 用 empty、full 控制容量，用 mutex 保護緩衝區。wait 順序若放反，可能持鎖休眠而造成 deadlock。
- readers-writers 要處理共享讀與互斥寫，並在公平性與讀者吞吐間取捨。dining philosophers 顯示資源取得順序如何造成循環等待。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.7　模型、時序與反例：經典同步問題

模型假設。 資源取得不是 atomic all-or-nothing，除非協定額外保證；等待者不會因公平政策被永久跳過。

不變量與時序。 buffer 內容與計數一致；philosopher 只有在取得兩支資源後進食，離開時全部釋放。

反例或邊界。 五位哲學家同時拿左叉再等右叉，若沒有順序限制會形成五邊等待環。

完整算例。 5 philosophers 的 table 有5 forks，模型限制最多4人同時嘗試。若每個人各持左叉，最多4位持有者、至少1隻額外叉可用；trace 中第4位取得第二叉後完成，沒有5人各持一叉的 state。

思考題。 deadlock-free 為何仍可能 starvation？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.7 bounded-buffer 的正確性條件

semaphore 解決三種不同條件：

- `empty` 阻止 producer 寫入已滿 buffer。
- `full` 阻止 consumer 讀取空 buffer。
- `mutex` 隔離槽位、head、tail 與 size 的複合更新。

資源等待必須在 `mutex` 外完成。持 mutex 等待 empty 時，consumer 可能先需要同一 mutex 才能取出項目，結果 producer 等 consumer 放容量，consumer 等 producer 放 mutex。

若 buffer 容量為 n，安全條件是 `0<=items<=n`。一般容量記帳要把在途 reservation 納入：令 E、F 分別為 empty、full 尚未保留的非負 free permits；P 是已獲授 empty permit、尚未完成 signal(full) 的 producer reservation 數；C 是已獲授 full permit、尚未完成 signal(empty) 的 consumer reservation 數。包含直接 handoff 給已喚醒但未執行者的授權，皆從授出時算入 P 或 C。於每個原子協議轉移邊界維持 `E+F+P+C=n`：producer 成功 wait 把 E 轉 P，signal(full) 把 P 轉 F 或直接轉給 consumer 的 C；consumer 對稱地把 F 轉 C，再把 C 轉 E 或 producer 的 P。資料寫入或取出本身不改這個容量分類。

5.7 bounded-buffer 的正確性條件 (續)

容量3、空 buffer 初始 `(E,F,P,C)=(3,0,0,0)`；producer 僅完成一次 wait(empty) 後為 `(2,0,1,0)`，即使尚未寫入，`E+F=2` 已不是3。只有沒有在途 reservation 的靜止邊界 `P=C=0`，才可寫 `E+F=n`。這裡 E/F 不是 signed semaphore 的 raw counter：raw 負值另含 queued waiters，不能直接代入 permit 守恆式；即使 raw 為0，也可能已有 handoff 授權在途。每次資料結構修改在 mutex 內完成，且先寫入再發布 full、先取出再發布 empty，才維持 occupancy 安全；任意時刻的 F 也不必等於 items。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.7 readers-writers 的兩種公平性

版本	進入規則	被保留的併發性	可能 starvation
First readers-writers	writer 已持有時 reader 等待；writer 只在等待時，新 reader 仍可加入	多 reader 同時讀	writer
Second readers-writers	一旦 writer 等待，新 reader 不得開始	writer 較快取得寫入機會	reader

第一種版本以 `read_count` 記錄讀者數：第一個 reader 取得 `rw_mutex`，最後一個 reader 釋放它。`mutex` 只保護 `read_count`。這種設計示範了為何鎖只保護它實際涵蓋的 state，並不能靠鎖的名稱描述語意。

5.7 dining philosophers 的互斥與死結

五位哲學家共用五支單筷。每人可先拿左再拿右：

```
wait(chopstick[i]);
wait(chopstick[(i+1)%5]);
eat;
signal(chopstick[(i+1)%5]);
signal(chopstick[i]);
```

若五人同時各拿左筷，五支 fork 都已被持有，所有人再等右筷，形成五節等待環。

常見修法有三種：

1. 最多四人同時嘗試，保證至少一人不會同時持有左筷並等待右筷。
2. 同時可取得兩支筷才進食。
3. 奇偶哲學家使用相反的取得順序。

修法能排除這個 deadlock cycle，但不自動提供 starvation-free 的公平排程。解法還必須規定等待者如何被選取。

5.8 Monitors

- monitor 把共享資料與操作封裝在同一同步抽象內，一次只允許一個執行流進入。condition variable 的 wait 與 signal 需要在正確 monitor 狀態下操作。
- condition variable 不保存 permit。通知若沒有等待者就可能消失，因此等待者要用 while 迴圈重檢 predicate。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.8 模型、時序與反例：Monitors

模型假設。 monitor 自動提供互斥；condition wait 原子釋放 monitor 並排入等待，signal 不保存 permit。

不變量與時序。 每次操作結束與每次 wait 釋放前，monitor 資料都對應到 predicate；被喚醒者重新檢查 predicate。

反例或邊界。 使用 `if` 而非 `while` 等待，遇到 spurious wakeup 後可能跳過 predicate 直接讀半成品。

完整算例。 buffer capacity=2。producer 放入 A、B 後 size=2；consumer 在 while 迴圈等到 size>0，取 A 後 size=1 並 notify，再取 B。若把 while 改成 if，額外喚醒或 predicate 被其他 consumer 消耗後可能對空 buffer 取值。

思考題。 signal 與 broadcast 的選擇如何影響正確性與喚醒成本？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.8 monitor 與 condition variable

monitor 把共享變數與操作封裝在同一個同步物件，執行流進入任一個 monitor operation 時自動取得 monitor lock。它移除手寫 entry/exit 的漏接，但呼叫 sequence 仍屬程式契約。

condition variable 只有 `wait` 與 `signal` 兩個概念操作：

- `x.wait()` 原子地釋放 monitor 並把呼叫者放入 x queue。
- `x.signal()` 若有人等待，恰好恢復一個；若沒有人等待，該次 signal 不保存。

CV 本身不是 predicate，也不是 semaphore。真正的條件是 monitor state，例如 `items>0`。等待者要在重新取得 monitor 後重檢 predicate。

5.8 Hoare 與 Mesa 的交接差異

語意	signal 後誰先執行	被喚醒者返回時	常搭配的等待形式
Hoare signal-and-wait	signaler 暫停，被喚醒者立即取得 monitor	邏輯條件在交接點保持成立	可用 `if`，仍需遵守該語言契約
Mesa signal-and-continue	signaler 繼續執行，被喚醒者進入 ready queue	取得 monitor 前可能被其他執行流改寫 predicate	必須以 `while` 重檢
Concurrent Pascal signal-and-exit	signaler 立即離開 monitor	被喚醒者接手	語言定義決定

POSIX condition variable 與 Java common implementations 不能靠 signal 保存 permit。即使沒有 spurious wakeup，多個 consumer 競爭同一項資料也會讓 predicate 在返回後改變。

5.8 lost wakeup 的兩種事件順序

不變量：`ready` 與等待都只透過同一個 mutex 協調。notifier 在 mutex 內令 `ready=true` 後 signal；waiter 在 mutex 內檢查條件。

Waiter 先。 waiter 看見 false，`wait(cv)` 原子釋放 mutex 並入 queue。notifier 之後取得 mutex，設定 true，signal 能找到該 waiter。

Notifier 先。 notifier 先設定 true 後離開；waiter 取得 mutex 時看見 true，不呼叫 wait。通知雖已過去，predicate 仍保存事實。

若 waiter 以 `if` 而不是 `while`，即使 lost wakeup 被排除，被額外喚醒或 predicate 被其他執行流消耗後，它仍可能直接使用不存在的資料。等待迴圈與 predicate mutex 一起構成證明。

5.9 Synchronization examples

- 教科書的 Windows 案例以 spinlock 保護 SMP 上短暫的 kernel critical section；kernel 不得在持有 spinlock 時被搶占。User mode 的 dispatcher objects 包含 mutex、semaphore、event 與 timer，狀態分為 signaled/nonsignaled；contended critical-section object 會先短暫 spin，再配置 kernel mutex 並讓出 CPU。
- Linux 核內提供 atomic integer、mutex、spinlock 與 semaphore。Linux 2.6 以後 kernel 可搶占；SMP 的短 critical section 使用 spinlock，較長等待使用可睡眠的 mutex/semaphore。User-level Pthreads 的 mutex、condition variable、reader-writer lock 則依 API contract 運作，不能直接混同核內鎖。
- Solaris 的 adaptive mutex 在 holder 正於另一 CPU 執行時 spin，在 holder sleeping 時 block。Turnstile 排隊 blocked threads，並支援 priority inheritance 以避免高優先權 thread 被低優先權 holder 長時間阻塞。

5.9　模型、時序與反例：Synchronization examples

模型假設。 各 OS/API 的 lock、signal、cancellation 與 fairness 語意依其版本；不可跨 API 直接搬用。

不變量與時序。 不論實作名稱，必須確認 mutex ownership、condition wakeup、interrupt priority 與 lock order 的組合。

反例或邊界。 在 interrupt handler 取一般 sleeping mutex，可能因不可睡眠上下文而死結或 panic。

完整算例。 Windows 7 dispatcher mutex M 初始為 signaled。T1 acquire 後 M 變 nonsignaled，T2 acquire 進入該 object 的 waiting queue；T1 release 使 M 變 signaled，核心只選一位 mutex waiter，T2 由 waiting 變 ready 並重新取得 M，object 又回到 nonsignaled。

思考題。 哪一個 API 保證在現代版本中仍需以官方文件重驗而非依書本推定？

5.9　固定版本的同步機制

下列是 *Operating System Concepts* 第 9 版案例的歷史快照。它們用來說明同一 primitive 在不同執行環境的成本與保證，不能直接代表今日所有版本。

案例	實際機制	等待策略與邊界
Windows 7 kernel	SMP 上以 spinlock 保護短的 kernel global resource	持 spinlock 時不讓 thread 被搶占
Windows 7 user mode	dispatcher objects 有 mutex、semaphore、event、timer	狀態為 signaled/nonsignaled；mutex 只喚醒一位 owner
Linux 2.6 起	可搶占核心；atomic_t、mutex、spinlock、semaphore	spinlock 用於 SMP 短區段；`preempt_count>0` 時不可安全搶占
Solaris 教材案例	adaptive mutex、reader-writer lock、turnstile	holder 正執行時可 spin，否則 sleep；turnstile 支援 priority inheritance
Pthreads/POSIX	mutex、condition variable、reader-writer lock；semaphore 屬 POSIX SEM	API 契約決定 blocking、wakeup 與錯誤回傳

5.9 為何同名 primitive 不能直接互換

Windows critical-section object 先在 user mode 短暫 spin，競爭過久才配置 kernel mutex。Linux 的 spinlock 不會在單核上承擔同樣的忙碌等待意義，該案例以停用 kernel preemption 取代。Solaris adaptive mutex 會依 holder 是否 running 決定 spin 或 sleep。

判斷是否可替換時，依序問：

1. 呼叫所在的是 user context、kernel context 還是 interrupt context。
2. 等待者可以是 process、thread 還是 task。
3. primitive 是否有 owner，是否允許跨 process 共享。
4. 是否保證 fairness、priority inheritance、cancellation 或 wakeup 順序。
5. 指定版本與 API 的文件是否提供相同的 memory ordering。

名稱相似不構成語意等價。

5.10 Alternative approaches

- transactional memory 把記憶體交易以 atomically commit 或 abort 處理，適合可重試且衝突不多的操作。
- message passing 與 immutable functional style 可減少共享可寫狀態；lock-free 結構仍有共享可寫狀態，只提供不同的進度保證。它們降低成本的位置不同，仍需處理飢餓與記憶體回收。

5.10　模型、時序與反例：Alternative approaches

模型假設。 替代方案有明確 conflict detection、retry 次數、記憶體回收與 progress guarantee。

不變量與時序。 成功 commit 後的資料可見性符合抽象原子操作；abort 不留部分更新。

反例或邊界。 lock-free queue 的 pop 若在回收節點後仍讓其他 thread 讀取，會發生 use-after-free，而不只是資料競爭。

完整算例。 初始 A=100、B=200。transaction T1 執行 A-=25、B+=25，若在 commit 前偵測到 read set 衝突就 abort，A/B 仍分別是 100/200；重試成功後才一次可見 A=75、B=225。一次 abort 與一次成功 commit 不得各自留下部分 debit/credit。

思考題。 transaction 的效率為何不能只看衝突率，還要看 commit 成本？

5.10　transactional memory

memory transaction 把一串 shared reads/writes 表示成全有或全無。成功時 commit，失敗時 abort 並 rollback；其他執行流不能看見半完成的 transaction。

```
atomic {
  x = x - 25
  y = y + 25
}
```

若 conflict detection 發現另一個 transaction 已改動 read set 或 write set，其中一個 abort 後重試。放棄整段 transaction 可消除鎖順序造成的 deadlock，但 transaction 仍可能 starvation，而且需要處理 I/O、不可逆副作用與重試成本。

software transactional memory 以 compiler/runtime instrumentation 記錄讀寫；hardware transactional memory 利用 cache-coherent 機制偵測衝突。兩者失敗路徑與可支援的區段大小不同。

5.10 OpenMP、immutability 與實際邊界

OpenMP 的 `#pragma omp critical` 把後續區段視為具名互斥區。相同名稱的區段一次只允許一個 thread；不同名稱代表不同 critical region。它能減少手寫 mutex boilerplate，仍要求程式設計者找出所有共享衝突。

functional programming 以 immutable values 消除同一位置的 in-place update。計算建立新值，使多個執行流能安全共享舊值。這不表示整個程式沒有共享資源；檔案、網路、mutable database 與 message mailbox 仍需同步或通訊協定。

message passing 以複製或轉移訊息避免共同 mutable memory，但 channel capacity、ordering、backpressure 與失敗重送仍是同步問題。方案的成本要放在實際邊界上比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.11 總結

- 正確同步要同時滿足資料不變量、進度與公平。atomic primitive 只提供起點，較高階構造要維持 predicate 與喚醒的正確關係。
- 測驗 race 需要交錯模型與 invariant check。單純看 final value 可能漏掉時序錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.11　模型、時序與反例：總結

模型假設。 同步分析要覆蓋所有存取、所有失敗與取消路徑；lock-free 仍需定義記憶體回收與 progress。

不變量與時序。 操作前後共享抽象狀態滿足不變量；等待者若存在且資源可得，最終有某執行流前進。

反例或邊界。 只在正常 return 釋放鎖，exception 後持有鎖會讓後續操作永久阻塞，平均測試可能不易觸發。

完整算例。 兩個 threads 各有 load、increment、store 三步，未同步時共有 C(6,3)=20 種保留各自順序的完整交錯。交錯 P1.load(0)、P2.load(0)、P1.store(1)、P2.store(1) 留下 counter=1；其餘若干交錯得到 1 或 2。加入同一 lock 後，兩個 read-modify-write 區段只能分成先 P1 後 P2 或先 P2 後 P1，兩種結果都是 2。

思考題。 如何設計涵蓋取消與例外路徑的同步驗證？

5.11　選 primitive 前的重建表

問題	必要資訊	應檢查
只有一個 critical section	臨界區長度、是否可 sleep、是否有 owner	lock acquire/release 是否成對
有固定數量 resource	capacity、初始 permits、誰可 signal	semaphore 是否可能超額 signal
等待 predicate	predicate 變數、通知條件、喚醒後重檢	CV 是否綁定同一 mutex
多個 locks	全域順序、動態 key、取消路徑	每條路徑是否仍依序取得
多 readers 與 writers	公平版本、writer 是否可等待	是否出現 writer 或 reader starvation

完成選型後，逐事件列出 mode、owned resources、waiting set 與 predicate。只在平均測試中看到正確答案，不能排除少見交錯；一般性證明與可重跑反例要分開提出。

本章收束

- critical section 三性質分別對應安全、活性與公平
- semaphore 與 monitor 的錯誤多在等待條件而非 API 語法
- 經典假設不必然適用弱記憶體或現代硬體

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

練習與參考資料

練習題與詳解分別見本章獨立 PDF。

- Operating System Concepts, 9th Edition，第 5 章，印刷頁 203-258。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與對照範圍

- Silberschatz, Galvin, and Gagne, *Operating System Concepts*, 9th edition, Chapter 5, printed pages 203-242.
  - 5.1 Background，printed pages 203-206：race condition、counter 交錯與 critical-section 結構。
  - 5.2 The Critical-Section Problem，printed pages 206-207：mutual exclusion、progress、bounded waiting。
  - 5.3 Peterson's Solution，printed pages 207-209：flag、turn、互斥與有限等待證明的前提。
  - 5.4 Synchronization Hardware，printed pages 209-212：interrupt masking、test-and-set、compare-and-swap 與 bounded-waiting 實作。
  - 5.5 Mutex Locks，printed pages 212-213：available、acquire、release 與 spinlock 成本。
  - 5.6 Semaphores，printed pages 213-218：兩種 semaphore 實作、deadlock、starvation、priority inversion。

參考資料與對照範圍 (續)

- Silberschatz, Galvin, and Gagne, *Operating System Concepts*, 9th edition, Chapter 5, printed pages 203-242.
  - 5.7 Classic Problems of Synchronization，printed pages 219-223：bounded-buffer、readers-writers、dining philosophers。
  - 5.8 Monitors，printed pages 223-232：condition variable、Hoare/Mesa 交接、monitor 實作與 dining philosopher monitor。
  - 5.9 Synchronization Examples，printed pages 232-238：Windows、Linux、Solaris、Pthreads 的固定教材案例。
  - 5.10 Alternative Approaches，printed pages 238-242：transactional memory、OpenMP、functional programming。
  - 5.11 Summary，printed page 242。

參考資料與對照範圍 (續)

本章新增的交錯表、程式狀態轉移、Mesa 重檢理由與 memory-order 邊界，是依上述機制重組的教學材料。Windows 7、Linux 2.6、Solaris 與 Pthreads 的敘述保留原案例的版本界線，不外推成所有現代系統的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study